LOYOLA ACADEMY DEGREE & PG COLLEGE

Old Alwal, Secunderabad-500010

(An Autonomous Degree College Affiliated to Osmania University)

Accredited By NAAC with 'A' Grade College with Potential for Excellence

PRACTICALRECORD

DEPARTMENT OF COMPUTER SCIENCE AND CYBER SECURITY
SUBJECT**: PYTHON SCRIPTING**

CERTIFICATE

This is to clarify that this is a bonafide record of the work done in **Python Scripting** Practicals during 2nd year 4th semester of the academic **year 2025-2026.**

Name:

UID:

Class:

Internal Examiner Principal External Examiner

INDEX

S.No	Topic	Page No	Date	Signature
1.	Programs on Regular Expressions			
2.	Programs using Numbers in Python			
3.	Programs on Arithmetic and Math Built-in functions			
4.	Program to implement Array Module			
5.	Programs on GUI Widgets with T-kinter			
6.	Programs on Checkboxes, Radio buttons and Dialog boxes			
7.	Programs to implement XML using SAX and HTML Parser			

1. Programs on Regular Expressions

a. Compile(string)

```
import re
p=re.compile("python")
print(p,type(p))
obj=re.match(p,"python is easy")
print(obj)
```

Output:-

```
re.compile('python') <class 're.Pattern'>
<re.Match object; span=(0, 6), match='python'>
```

b. Match(pattern, target string)

```
import re
s=input("enter a pattern to check:")
m=re.match(s,"abcadefg")
if m!=None:
print("match found at")
print("start index:",m.start(),"and ending index:",m.end())
else:
print("match not found")
```

Output:-

```
enter a pattern to check:abcadefg
match found at
start index: 0 and ending index: 8
```

```
enter a pattern to check:abcabdefg
match not found
```

c. Fullmatch(pattern, target string)

```
import re
s=input("enter a pattern to check:")
m=re.fullmatch(s,"ababab")
if m!=None:
print("full string is matched")
else:
print("full string is not matched")
```

Output:-

```
enter a pattern to check:ababab
full string is matched

enter a pattern to check:abaaba
full string is not matched
```

d. Search(pattern, target string)

```
import re
s=input("enter a pattern to check:")
m=re.search(s,"abaaaba")
if m!=None:
print("match is available:")
print("starting index:",m.start(),"and ends at:",m.end())
else:
print("match is not available:")
```

Output:-

```
enter a pattern to check:abaaaba
match is available:
starting index: 0 and ends at: 7

enter a pattern to check:abaaaaba
match is not available:
```

```
import re
s="welcome to cyber security department"
m=re.search("cyber",s)
print("starting index:",m.start())
print("ending index:",m.end())
print("strings:",m.group())
```

Output:-

```
starting index: 11
ending index: 16
strings: cyber
```

e. Sub(pattern, new string, target string)

```
import re
s=re.sub("[a-z]","#","a7b9c5k8z")
print(s)
```

Output:-

```
#7#9#5#8#
```

f. Subn(pattern, new string, target string)

```
import re
s=re.subn("[a-z]","#","a7b9c5k8z")
print(s)
print("The result string:",s[0])
print("The number of replacement:",s[1])
```

Output:-

```
('#7#9#5#8#', 5)
The result string: #7#9#5#8#
The number of replacement: 5
```

g. Split(delemeter, string)

```
import re
l=re.split(",","sumy,bunny,chimy,vinny,pimy")
print(l)
```

Output:-

```
['sumy', 'bunny', 'chimy', 'vinny', 'pimy']
```

```
import re
l=re.split(",","sumy,bunny,chimy,vinny,pimy")
for i in l:
    print(i)
```

Output:-

```
sumy
bunny
chimy
vinny
pimy
```

```
import re
l=re.split(",","www.google.com")
for i in l:
    print(i)
```

Output:-

```
www.google.com
```

```
import re
l=re.split(",","www.google.com")
print(l)
```

Output:-

```
['www.google.com']
```

h. Findall(pattern, target string)

```
import re
l=re.findall("[0-9]","a75bc5kz3")
print(l)
```

Output:-

```
['7', '5', '5', '3']
```

```
import re
string="today I am going to watch the match"
p="[a-m]"
result=re.findall(p,string)
print(result)
```

Output:-

```
['d', 'a', 'a', 'm', 'g', 'i', 'g', 'a', 'c', 'h', 'h', 'e', 'm', 'a', 'c', 'h']
```

i. Finditer(pattern, target string)

```
import re
itr=re.finditer("[a-z]","a7bbc5k8z")
for i in itr:
   print(i.start(),".....",i.end(),"....",i.group())
```

Output:-

```
0 ..... 1 .... a
2 ..... 3 .... b
3 ..... 4 .... b
4 ..... 5 .... c
6 ..... 7 .... k
8 ..... 9 .... z
```

j. Does a string start with any special string? (^ symbol)

```
import re
s="brown fox jumping over the lazy cat"
res=re.search("^brown fox",s)
if res!=None:
print("string start with brown fox")
else:
print("not started")
```

Output:-

```
string start with brown fox
```

k. Does a string end with any special string? ($ symbol)

```
import re
s="brown fox jumping over the lazy cat"
res=re.search("lazy cat$",s)
if res!=None:
print("string ends with lazy cat")
else:
print("Does not end")
```

Output:-

```
string ends with lazy cat
```

2. Program using Numbers in Python

```
x=10
y=5
c=2+3j
d=2.5+2j
print(x+y)
print(x-y)
print(x/y)
print(x//y)
print(x%y)
print(x**2)
print(int(5.8))
print(float(2))
print(abs(-2.4))
print(bin(4))
print(hex(4))
print(oct(4))
```

Output:-

```
15
5
2.0
2
0
100
5
2.0
2.4
0b100
0x4
0o4
```

```
x=10
y=5
c=2+3j
d=2.5+2j
print(complex(1,-2.3))
print(c.real)
print(c.imag)
print(c.conjugate())
print(c+d)
print(c-d)
print(c*d)
print(c/d)
```

Output:-

```
(1-2.3j)
2.0
3.0
(2-3j)
(4.5+5j)
(-0.5+1j)
(-1+11.5j)
(1.0731707317073174+0.34146341463414637j)
```

```
import random
print(random.randint(1,10))
print(random.uniform(1,20))
print(random.randrange(1,10,2))
```

Output:-

```
4
14.600251914526762
3
```

3. Programs on Arithmetic and Math Built-in functions

a. Arithmetic Operations

```
a = 10
b = 3
print("Addition:", a + b)
print("Subtraction:", a - b)
print("Multiplication:", a * b)
print("Division:", a / b)
print("Floor Division:", a // b)
print("Modulus:", a % b)
print("Exponent:", a ** b)
```

Output:-

```
Addition: 13
Subtraction: 7
Multiplication: 30
Division: 3.3333333333333335
Floor Division: 3
Modulus: 1
Exponent: 1000
```

b. Comparison Operations

```
print("a > b:", a > b)
print("a < b:", a < b)
print("a == b:", a == b)
print("a != b:", a != b)
```

Output:-

```
a > b: True
a < b: False
a == b: False
a != b: True
```

c. Built-in Math Functions

```
print("abs(-5):", abs(-5))
print("pow(2, 3):", pow(2, 3))
print("round(3.6):", round(3.6))
print("max(1, 5, 3):", max(1, 5, 3))
print("min(1, 5, 3):", min(1, 5, 3))
```

```
print("sum([1, 2, 3]):", sum([1, 2, 3]))
```

Output:-

```
abs(-5): 5
pow(2, 3): 8
round(3.6): 4
max(1, 5, 3): 5
min(1, 5, 3): 1
sum([1, 2, 3]): 6
```

d. Math Module Constants

```
import math
print("Pi:", math.pi)
print("e:", math.e)
```

Output:-

```
Pi: 3.141592653589793
e: 2.718281828459045
```

e. Math Module Functions

```
print("sqrt(16):", math.sqrt(16))
print("factorial(5):", math.factorial(5))
print("floor(3.7):", math.floor(3.7))
print("ceil(3.2):", math.ceil(3.2))
print("gcd(12, 18):", math.gcd(12, 18))
```

Output:-

```
sqrt(16): 4.0
factorial(5): 120
floor(3.7): 3
ceil(3.2): 4
gcd(12, 18): 6
```

f. Trigonometric Functions

```
angle = math.pi / 2
print("sin(90°):", math.sin(angle))
print("cos(90°):", math.cos(angle))
print("tan(45°):", math.tan(math.pi / 4))
```

Output:-

```
sin(90°): 1.0
cos(90°): 6.123233995736766e-17
tan(45°): 0.9999999999999999
```

g. Logarithmic Functions

```
print("log(10):", math.log(10))
print("log10(100):", math.log10(100))
```

Output:-

```
log(10): 2.302585092994046
log10(100): 2.0
```

h. Type Conversion Functions

```
print("int(3.9):", int(3.9))
print("float(5):", float(5))
print("complex(2, 3):", complex(2, 3))
```

Output:-

```
int(3.9): 3
float(5): 5.0
complex(2, 3): (2+3j)
```

i. Pythagoras Theorem

```
hypotenuse = math.sqrt(a**2 + b**2)
print("Side a:", a)
print("Side b:", b)
print("Hypotenuse:", hypotenuse)
```

Output:-

```
Side a: 10
Side b: 3
Hypotenuse: 10.44030650891055
```

j. Infinity Function

```
x = math.inf
y = -math.inf
```

```
print(x > 1000)
print(y < -1000)
```

Output:-

```
True
True
```

4. Program to implement Array Module

a. Creating Array and Displaying Elements

```
import array as arr
a=arr.array('i',[1,2,3,4,5,6])
print(a[0])
print(a[3])
print(*a)
b=arr.array('d',[4.5,7,0.7,1.5,8])
print(b[1])
print(*b)
```

Output:-

```
1
4
1 2 3 4 5 6
7.0
4.5 7.0 0.7 1.5 8.0
```

```
a=arr.array('i',[1,2,3,4,5,6])
for i in range(0,6):
    print(a[i],end=" ")
```

Output:-

```
1 2 3 4 5 6
```

b. Adding Elements to an Array

```
a=arr.array('i',[1,2,3,4,5,6])
a.insert(4,7)
print(*a)
```

Output:-

```
1 2 3 4 7 5 6
```

c. Removing Elements from an Array

```
a=arr.array('i',[1,2,3,4,5,6])
a.remove(3)
print(a)
a.pop(4)
print(a)
```

Output:-

```
array('i', [1, 2, 4, 5, 6])
array('i', [1, 2, 4, 5])
```

d. Slicing an Array

```
a=[1,2,3,4,5,6,7,8,9,10]
b=arr.array('i',a)
res=a[3:8]
print(res)
res=a[5:]
print(res)
res=a[:]
print(res)
```

Output:-

```
[4, 5, 6, 7, 8]
[6, 7, 8, 9, 10]
[1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
```

e. Searching an Element in an Array

```
a=arr.array('i',[1,2,3,4,5,6])
print(a.index(3))
print(a.index(4))
```

Output:-

```
2
3
```

f. Updating an Element in an Array

```
a=arr.array('i',[1,2,3,4,5,6])
a[2]=8
print(*a)
```

Output:-

```
1 2 8 4 5 6
```

g. Count Method

```
a=arr.array('i',[1,3,3,4,5,6])
count=a.count(3)
print(count)
```

Output:-

2 \

h. Reverse Method

```
a=arr.array('i',[1,2,3,4,5,6])
a.reverse()
print(*a)
```

Output:-

6 5 4 3 2 1

i. Sorting Elements in an Array

```
a=arr.array('i',[1,2,9,4,5,8])
print(sorted(a))
```

Output:-

[1, 2, 4, 5, 8, 9]

j. Extending Elements into an Array

```
import array as arr
a=arr.array('i',[1,2,3,4,5,6])
a.extend([7,8,9])
print(*a)
```

Output:-

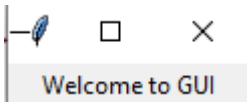
1 2 3 4 5 6 7 8 9

5. Programs on GUI Components

a.Program to create a Label

```
from tkinter import *
root=Tk()
lb=Label(root,text="Welcome to GUI")
lb.pack()
root.mainloop()
```

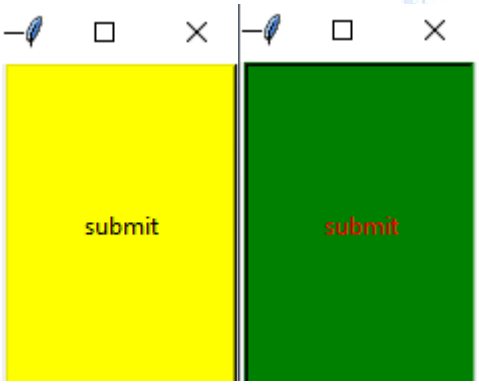
Output:-



b. Program to create a Button

```
from tkinter import *
root=Tk()
b=Button(root,text="submit",width=15,height=10,bg="yellow",activebackground="green",activeforegroun
d="red")
b.pack()
root.mainloop()
```

Output:-

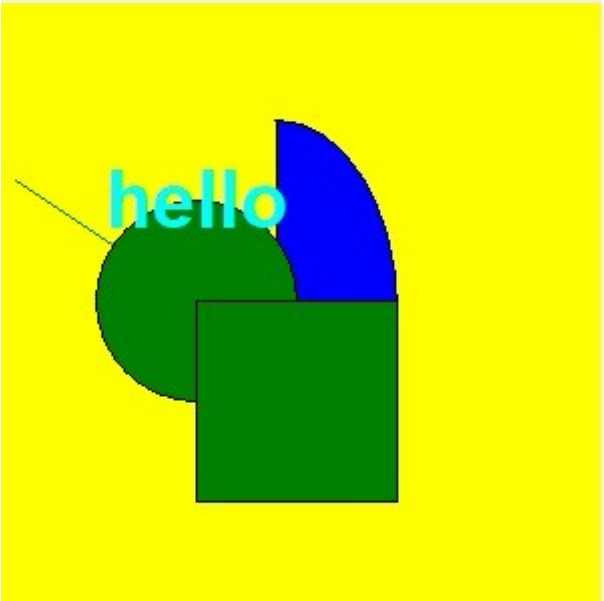


c. Program to create a Canvas

```
import tkinter
from tkinter import *
root=Tk()
root.title("canvas widget")
root.geometry("300x300")
c=Canvas(root,height=300,width=300,bg="yellow")
line=c.create_line(10,90,100,150,fill="green")
arc=c.create_arc(80,60,200,250,fill="blue")
```

```
oval=c.create_oval(50,100,150,200,fill="green")
rect=c.create_rectangle(100,150,200,250,fill="green")
txt=c.create_text(100,100,text="hello",font=("helvetica",30,'bold'),fill="cyan")
c.pack()
root.mainloop()
```

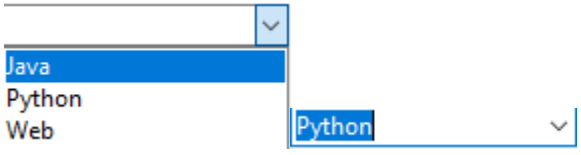
Output:-



d. Program to create a Dropdown Menu

```
from tkinter import *
from tkinter import ttk
choices=["java","python","web"]
root=Tk()
root.geometry('300x300')
dropdown=ttk.Combobox(root,values=choices)
dropdown.pack()
root.mainloop()
```

Output:-



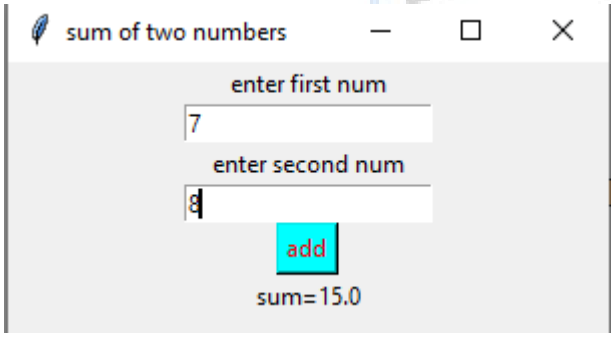
e. Program to create an Entry (Sum of 2 Numbers)

```
import tkinter as tk
def add_num():
```

```
    try:
        n1=float(e1.get())
        n2=float(e2.get())
        total=n1+n2
        lb.config(text=f"sum={total}")
    except ValueError:
        lb.config(text="invalid inputs")

root=tk.Tk()
root.title("sum of two numbers")
root.geometry('300x300')
lb1=tk.Label(root,text="enter first num")
lb1.pack()
e1=tk.Entry(root)
e1.pack()
lb2=tk.Label(root,text="enter second num")
lb2.pack()
e2=tk.Entry(root)
e2.pack()
b=tk.Button(root,text="add",bg='cyan',fg="red",command=add_num)
b.pack()
lb=tk.Label(root,text="the sum is")
lb.pack()
root.mainloop()
```

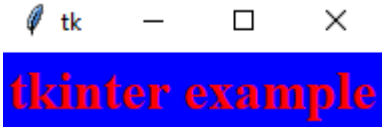
Output:-



f. Program to create a Font

```
import tkinter as tk
from tkinter import *
from tkinter import font
root=tk.Tk()
lbl=tk.Label(root,text="tkinter example",fg="red",bg="blue",font=("times new
roman",20,"bold")).pack(expand=YES,fill=BOTH)
#fontlist=list(tk.font.families())
#print(fontlist)
root.mainloop()
```

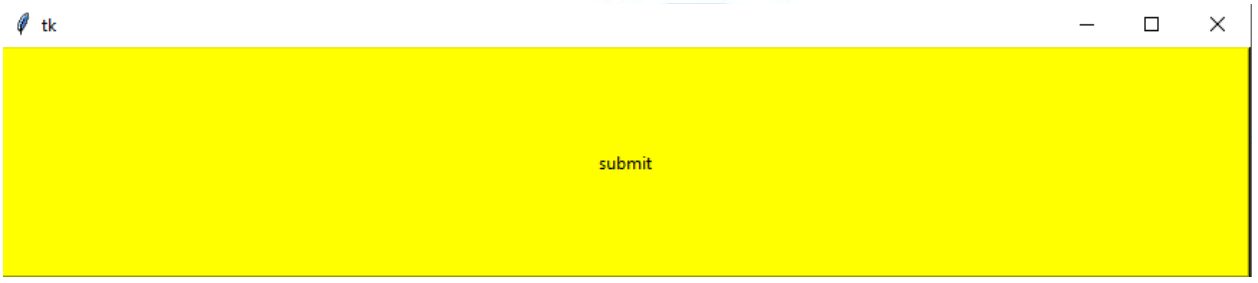
Output:-



g. Program to Resize the Widget

```
from tkinter import *
root=Tk()
b=Button(root,text="submit",width=15,height=10,bg="yellow",activebackground="green",activeforegroun
d="red")
b.pack(expand=YES,fill=BOTH)
root.mainloop()
```

Output:-

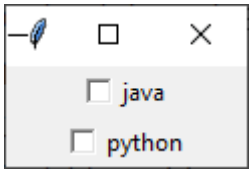


6. Programs on Checkboxes, Radio-buttons and Dialogboxes

a. Program to create Checkbox

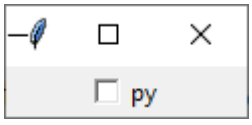
```
import tkinter
from tkinter import*
root=Tk()
v1=IntVar()
v2=IntVar()
ch1=Checkbutton(root,text="java",variable=v1)
ch1.pack()
ch2=Checkbutton(root,text="python",variable=v2)
ch2.pack()
root.mainloop()
```

Output:-




```
import tkinter
from tkinter import*
root=Tk()
var=BooleanVar()
ch1=Checkbutton(root,text="py",variable=var)
ch1.pack()
root.mainloop()
```

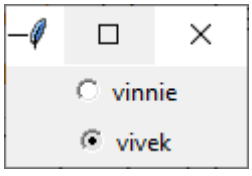
Output:-



b. Program to create Radiobutton

```
import tkinter
from tkinter import*
root=Tk()
r1=Radiobutton(root,text="vinnie",value=6).pack()
r2=Radiobutton(root,text="vivek",value=9).pack()
root.mainloop()
```

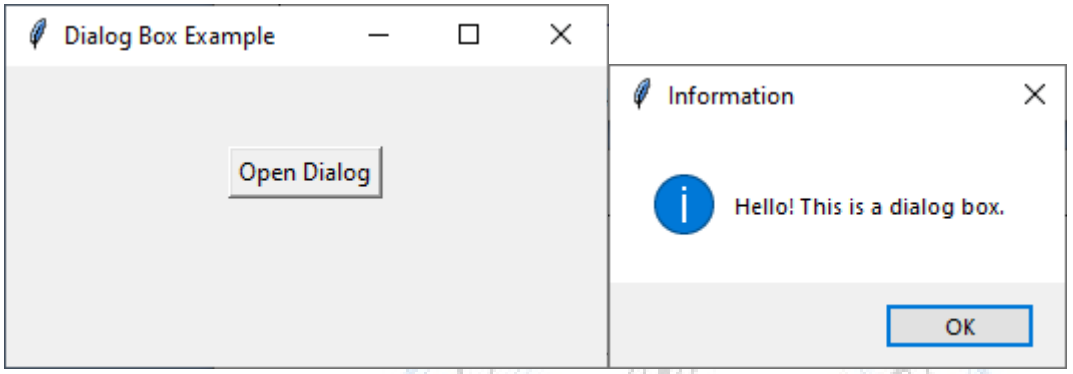
Output:-



c. Program to create Dialogboxes

```
from tkinter import messagebox
def show_dialog():
messagebox.showinfo("Information", "Hello! This is a dialog box.")
root = Tk()
root.title("Dialog Box Example")
root.geometry("300x150")
btn = tk.Button(root, text="Open Dialog", command=show_dialog)
btn.pack(pady=40)
root.mainloop()
```

Output:-



7. Programs to implement XML using SAX and HTML Parser

a. HTML Parser

```
from html.parser
 import HTMLParser
class Parser(HTMLParser):
    def handle_starttag(self, tag, attrs):
        print(f"Tag: {tag}")
    def handle_data(self, data):
        if data.strip():
            print(f"Data: {data}")

xml = """
<students>
  <student>
    <name>John</name>
    <age>20</age>
  </student>
  <student>
    <name>Sara</name>
    <age>19</age>
  </student>
</students>
"""

parser = Parser()
parser.feed(xml)
```

Output:-

```
Tag: students
Tag: student
Tag: name
Data: John
Tag: age
Data: 20
Tag: student
Tag: name
Data: Sara
Tag: age
Data: 19
```

b. DOM parser

```
from xml.dom import minidom
doc=minidom.parse("sample.xml")
root=doc.documentElement
print("Root element",root.tagName)
students=doc.getElementsByTagName("student")
for student in students:
    name=student.getElementsByTagName("name")[0].firstChild.nodeValue
    age=student.getElementsByTagName("age")[0].firstChild.nodeValue
    print("Name:",name,"Age:",age)
```

……

sample.xml

```
<students>
<student>
<name>Alice</name>
<age>20</age>
</student>
<student>
<name>bob</name>
<age>22</age>
</student>
<student>
<name>Arun</name>
<age>24</age>
</student>
</students>
```

Output:-

```
[Running] python -u "c:\Users\user\Downloads\dom.py"
Root element students
Name: Alice Age: 20
Name: bob Age: 22
Name: Arun Age: 24
```

c. Sax Parser

```
import xml.etree.ElementTree as ET
xml_data="""
<person>
<name>sai</name>
<city>hyd</city>
</person>
"""
root=ET.fromstring(xml_data)

print(root.find("name").text)
print(root.find("city").text)
```

Output:-

```
[Running] python -u "c:\Users\user\Downloads\sax.py"
sai
hyd
```

d. Sax Parser 2

```
import xml.etree.ElementTree as ET

tree=ET.parse("lib.xml")
root=tree.getroot()
for book in root:
    title=book.find("title").text
    author=book.find("author").text
print(title,author)
```

……..

lib.xml

```
<?xml version="1.0"?>
<library>
    <book>
        <title>python valume1</title>
        <author>rema</author>
    </book>
    <book>
        <title>python valume2</title>
        <author>nagesh</author>
        <author>nagesh rao</author>
    </book>
    <book>
        <title>python valume3</title>
        <author>chun</author>
    </book>
</library>
```

Output:-

```
[Running] python -u "c:\Users\user\Downloads\sax2.py"
python valume1 rema
python valume2 nagesh
python valume3 chun
```